



Cardiff
Metropolitan
University

Prifysgol
Metropolitan
Caerdydd

Cardiff Metropolitan University

Cardiff School of Technology

BSc Software Engineering

**AI-Based Personalized Nutrition, Meal Recommendation and
Weight Management System for Gym Users**

Submitted in August

2026

By

U.L.S.R. Perera

(CL.BSCSD.34.69 - 20360408 - ICBT)

This dissertation is submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Software Engineering (BSc SE)

DECLARATION

This work is being submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering (BSc SE) and has not previously been accepted in substance for any degree and is not being concurrently submitted in candidature for any degree.

Signed -

Date -

(Candidate)

SUPERVISOR'S DECLARATION STATEMENT

Student Name - U.L.S.R. Perera

Supervisor's Name -

I acknowledge that the above named student has regularly attended the meeting, and actively engaged in the dissertation supervision process.

Signed -

Date -

(Supervisor)

Acknowledgement

I would like to thank my supervisor for the guidance and feedback given throughout this project. The regular meetings kept the work on track and helped me judge what was realistic to build in the time available.

I also thank the lecturers at ICBT Campus and Cardiff Metropolitan University for the knowledge I applied directly in this project, and my family and friends for their support and patience.

Abstract

Gym users need to know how many calories and how much protein they should eat each day. Most people use free online calculators or generic diet plans. These give the same answer to everyone with the same height and weight, and they do not follow the user as their body changes. Getting personal advice from a dietitian is accurate but costs money and is not always available.

This project built NutriFit-AI, a working web application that solves this problem. The system takes a user's age, gender, height, weight, training habits and fitness goal, and predicts their daily calorie and protein needs using machine learning. It then recommends real meals that fit those numbers, builds a full eight-week meal plan, and recalculates everything when the user logs a new weight.

The system is built as three connected services: a React front end, a Node.js and Express API, and a Python FastAPI machine learning service, with Supabase PostgreSQL for storage and authentication. Two models were trained on 973 real gym member records: Linear Regression as a baseline and Random Forest as the main model. Random Forest performed better on both targets, with a cross-validated mean absolute error of 96.16 kilocalories and 8.72 grams of protein.

A learning curve experiment showed something more interesting than the headline numbers. Random Forest was worse than Linear Regression on small samples and only overtook it as the dataset grew, crossing over at about 680 records. This is evidence for why the model was chosen, not just a claim. The eight-week planner keeps every week within 0.4 percent of the calorie target, and 263 automated tests pass across all layers. The food catalogue was also extended with 52 Sri Lankan dishes after an audit found the public datasets contained almost no local food.

Table of Contents

Acknowledgement	3
Abstract.....	4
Table of Contents	5
List of Figures.....	8
List of Tables	9
Glossary	10
1.0 Introduction	11
1.1 Background Studies.....	11
1.2 Problem Statement.....	11
1.3 Objective.....	12
1.4 Solutions	12
2.0 Literature Review	13
2.1 Review of Existing Systems.....	13
2.2 System Architecture and Design Patterns	13
2.3 AI as an Integrated Feature.....	14
2.4 Recommendation Techniques.....	14
2.5 Security and Data Protection	15
2.6 Research Gap.....	15
3.0 Planning.....	16
3.0.1 Feasibility Report	16
3.0.2 Risk Assessment.....	16
3.0.3 SWOT.....	17
3.0.4 PESTEL Analysis.....	17
3.0.5 Life Cycle Model.....	17

3.1.1 Time Plan.....	18
4.0 Requirement Gathering and Analysis.....	19
4.2 Requirement Gathering and Analysis.....	19
4.2.1 Requirement Gathering technique used for the project.....	19
4.2.2 Questionnaire.....	19
4.2.3 Interview.....	20
Summary of the Interview and Questionnaire.....	21
4.3 Functional and Non-Functional Requirements.....	21
4.3.1 Functional Requirements.....	21
4.3.2 Non-functional Requirement.....	22
5.0 System Design.....	23
5.1 Architecture Diagram.....	23
5.2 ER Diagram.....	24
5.3 UML Diagrams.....	25
5.4 Wireframe Diagram.....	27
6.0 Implementation.....	29
6.1 Technology Stack.....	29
6.2 Design patterns.....	29
6.3 Implementation of the program.....	30
6.3.1 Datasets used.....	30
6.3.2 Building the training targets.....	31
6.3.3 Preprocessing pipeline.....	32
6.3.4 Recommendation engine.....	33
6.3.5 Meal plan generator.....	33
6.3.6 Cultural bias in the food catalogue.....	34

6.3.7 API design	34
7.0 Testing and Validation	35
Test Plan	35
7.1 Model Evaluation Results.....	35
7.2 The Learning Curve Experiment	36
7.3 Feature Importance	38
7.4 Meal Plan Validation	38
Test Cases	39
7.5 A Bug Found by Testing	39
7.6 What Was Not Tested.....	40
8.0 Conclusion	41
8.1 Conclusion	41
8.2 Future Recommendations	41
8.3 Lessons Learned	42
Gantt Chart	43
References	44
Appendix 1	45
Questionnaire.....	45
Interview Guide	46
Test Case	46
Supervision meeting log sheets	46

List of Figures

- Figure 5.1 System architecture
- Figure 5.2 Entity relationship diagram
- Figure 5.3 Use case diagram
- Figure 5.4 Sequence diagram for logging weight and adjusting the plan
- Figure 5.5 Machine learning pipeline data flow
- Figure 5.6 Wireframes: authentication and profile setup
- Figure 5.7 Wireframes: dashboard and meal recommendations
- Figure 5.8 Wireframes: two-month meal plan and progress
- Figure 6.1 Distribution of the model input features
- Figure 6.2 Correlation between features
- Figure 6.3 Distribution of the constructed calorie and protein targets
- Figure 6.4 Content-based recommendation and slot composition
- Figure 7.1 Predicted against actual values for both models and targets
- Figure 7.2 Learning curves for both models and both targets
- Figure 7.3 Random Forest feature importance
- Figure 7.4 Weekly meal plan error against the $\pm 5\%$ tolerance
- Figure 8.1 Project time plan (Gantt chart)

List of Tables

Table 3.1 Risk assessment and mitigation

Table 3.2 SWOT analysis

Table 3.3 PESTEL analysis

Table 3.4 Project phases

Table 4.1 Structure of the questionnaire

Table 4.2 Interview topics and the design question each addresses

Table 4.3 Functional requirements and where they are implemented

Table 4.4 Non-functional requirements

Table 6.1 Technology stack

Table 6.2 Design patterns used

Table 6.3 Backend API endpoints

Table 7.1 Automated test summary

Table 7.2 Model comparison on the held-out test set and by cross-validation

Table 7.3 Cross-validated MAE against training set size (calorie target)

Table 7.4 Eight-week meal plan accuracy

Table 7.5 Selected test cases

Table A1 Questionnaire response summary sheet

Table A2 Interview response recording sheet

Glossary

Term	Meaning
API	Application Programming Interface. A defined way for one program to call another.
BMI	Body Mass Index. Weight in kg divided by height in metres squared.
BMR	Basal Metabolic Rate. Energy the body uses at rest.
Cross-validation	Training repeatedly on several data folds, so results do not rest on one split.
Deurenberg equation	Estimates body fat percentage from BMI, age and sex.
FastAPI	Python web framework used here to serve the models.
Feature	One input column given to a model, such as weight.
Joblib	Python library used to save and reload a trained model.
JWT	JSON Web Token. A signed token proving a user is logged in.
Katch-McArdle equation	BMR formula based on lean body mass.
Label	The value a model is trained to predict; also called the target.
Linear Regression	Model fitting a straight-line relationship to the target.
MAE	Mean Absolute Error. Average error size, in the target's units.
Mifflin-St Jeor equation	BMR formula using weight, height, age and sex.
MSE / RMSE	Mean Squared Error and its root. Both punish large errors.
Microservice	A small independent service called over the network.
PAL	Physical Activity Level. Multiplier applied to BMR.
Pipeline	A scikit-learn object chaining preprocessing and a model.
R-squared	Share of variation the model explains. 1.0 is perfect.
Random Forest	Model averaging many decision trees.
RLS	Row Level Security. Database rules deciding which rows a user sees.
Supabase	Hosted PostgreSQL database with user authentication.
TDEE	Total Daily Energy Expenditure. BMR times activity level.
Train-serve skew	When training and prediction prepare data differently, making live results wrong.
WCAG	Web Content Accessibility Guidelines.

1.0 Introduction

Exercise alone does not produce results. What a person eats decides whether they lose fat, gain muscle, or stay the same. Gym users know this, but most of them do not know how much they should actually be eating. They guess, or they copy a plan from social media that was written for somebody else.

This project is a web application called NutriFit-AI. It predicts a gym user's daily calorie and protein needs from their own body and training data, recommends meals that match those numbers, and builds a two-month meal plan. When the user's weight changes, the system notices and offers to update the plan.

1.1 Background Studies

Daily energy needs are normally worked out in two steps. First the Basal Metabolic Rate (BMR) is estimated, which is the energy the body uses at rest. Then it is multiplied by an activity factor to get Total Daily Energy Expenditure (TDEE). The most common BMR equation is Mifflin-St Jeor (1990), which uses weight, height, age and sex.

For people who train, there is a better option. The Katch-McArdle equation calculates BMR from lean body mass instead. This matters because two people who weigh 85 kilograms but have very different body fat percentages burn different amounts of energy at rest, yet Mifflin-St Jeor gives them the same answer. Protein needs are usually given as grams per kilogram of body weight. The International Society of Sports Nutrition recommends 1.4 to 2.0 g/kg for people training, and more during a calorie deficit to protect muscle.

These formulas are well established, which raises a fair question: if they already exist, why use machine learning? Chapter 2 discusses this and Chapter 7 tests it.

1.2 Problem Statement

The main problem is that gym users do not have access to nutrition advice that is personal to them, adapts over time, and is free. The specific gaps are:

- Online calculators give a calorie number and stop there. They do not tell the user what to actually eat.
- Generic meal plans do not match the user's own calorie and protein targets.

- Existing apps focus on logging food after it is eaten, not on planning it in advance.
- Nothing recalculates automatically when the user's weight changes.
- Food databases in these apps are built around Western food, so the suggestions do not suit a Sri Lankan user.

1.3 Objective

The aim of this project is to design, build and test a working web application that gives gym users personalised nutrition guidance. The objectives are:

- To collect suitable public datasets and prepare them for machine learning.
- To build and compare two prediction models, Linear Regression and Random Forest, for daily calorie and protein needs.
- To evaluate both models properly using MAE, MSE, RMSE and R-squared, with cross-validation rather than a single split.
- To build a content-based recommendation engine that suggests real meals matching the predicted targets.
- To generate a structured eight-week meal plan with variety and accuracy controls.
- To build a secure web application around all of this, with user accounts, a database and a dashboard.
- To test the system at unit, integration and API level.

1.4 Solutions

The solution is a three-tier web system. The React front end handles the user interface. A Node.js and Express API handles authentication, validation and business rules. A separate Python FastAPI service holds the machine learning models and the recommendation engine. Supabase provides PostgreSQL storage and user authentication.

Splitting the machine learning into its own service was a deliberate decision. The models are written in Python because scikit-learn only runs in Python, while the web layer stays in Node.js. The two communicate over internal HTTP. The machine learning service is never exposed to the internet.

2.0 Literature Review

This is a Software Engineering development project, so this review concentrates on existing systems, the software architecture and design decisions behind them, and how artificial intelligence is added to a system as a feature. The machine learning is one component of the system, not the whole project.

2.1 Review of Existing Systems

MyFitnessPal is the best known nutrition app. Its strength is a very large food database and barcode scanning. Its weakness for this project is that it is a logging tool. The user tells it what they ate. It has a goal calculator, but it does not plan meals ahead or adjust the plan automatically as the user's body changes.

Cronometer is more accurate on micronutrients and uses verified data sources, but again it records rather than recommends.

MacroFactor is closer to this project. It adjusts calorie targets based on logged weight and intake over time, which is the same adaptive idea used here. However it is a paid subscription app, it still does not generate a full multi-week meal plan, and its food database is not adapted to South Asian diets.

Free online TDEE calculators are the tool gym users most often reach for. They implement Mifflin-St Jeor directly, but produce one number with no meal guidance, no storage and no follow-up.

Across these systems the gap is the same: calorie prediction, meal recommendation, long-term planning and automatic readjustment exist separately but rarely in one free system, and almost never with a food database suited to Sri Lankan users.

2.2 System Architecture and Design Patterns

For a system that mixes a web application with machine learning, two architectures are common. A monolith puts everything in one deployable unit, which is simple but forces one language. A microservice split puts the machine learning behind its own API.

This project uses the second approach, which is standard practice for serving scikit-learn models. The model is trained offline, saved with joblib, and loaded by a small stateless service. The web backend calls it like any other internal API. This keeps the machine learning in Python and the web logic in Node.js, and lets each part be tested separately.

The design patterns used are listed in Chapter 6.

2.3 AI as an Integrated Feature

In a Software Engineering project the AI must be judged on how well it fits into the system, not only on its accuracy score. Three integration problems matter in practice.

The first is train-serve skew: when preparation during training differs from preparation at prediction time, the output is quietly wrong. The fix is to wrap preprocessing and model in one saved object, which this project does.

The second is graceful degradation. If a model file cannot load the system should still answer, so this project falls back to the formula and records which method was used.

The third is that a model gives a number, not an action. Predicting 2,400 kilocalories does not help a user who does not know what to cook, so the recommendation engine turns it into meals.

2.4 Recommendation Techniques

Recommender systems are usually grouped into collaborative filtering, content-based filtering, and hybrids. Collaborative filtering needs rating history from many users, which a new system does not have. This is the cold-start problem.

Content-based filtering compares the features of the items themselves, so it works from the first day. For meal recommendation the features are the macronutrients. This project represents each food as a vector of calories, protein, carbohydrate, fat and fibre, and finds the nearest match to the user's remaining budget for that meal slot.

2.5 Security and Data Protection

Body weight and body fat percentage are personal health data, so access control matters. Application-level checks are easy to get wrong: one missing filter in one query can expose another user's rows.

PostgreSQL Row Level Security moves this enforcement into the database. Policies are attached to the tables themselves, so even if the application forgets a check, the database returns nothing. This project enables Row Level Security on all six tables.

2.6 Research Gap

From this review, the gap this project addresses is a single free system that predicts nutritional needs with machine learning, converts those predictions into real meal recommendations, produces a structured eight-week plan, adjusts itself when the user's weight changes, and uses a food catalogue that includes local Sri Lankan food.

3.0 Planning

3.0.1 Feasibility Report

Technical feasibility. Every tool used is free and well documented: Python, scikit-learn, React, Node.js, FastAPI and Supabase. Training was done in Google Colab. One useful finding here is that scikit-learn runs on CPU only, so a GPU runtime gives no benefit for Linear Regression or Random Forest. The entire training pipeline runs in about ten minutes on the free CPU runtime and used zero paid compute units.

Operational feasibility. The user enters a few fields once, then logs their weight occasionally. Everything else is automatic.

Economic feasibility. Total cost was zero. All datasets are public and all services used are free tiers.

Schedule feasibility. The work was planned across twelve weeks as shown in the time plan in section 3.1.1.

Legal and ethical feasibility. The datasets are public and contain no personal identifiers. The application shows a medical disclaimer stating that it gives nutritional guidance and does not replace professional medical advice.

3.0.2 Risk Assessment

Table 3.1 Risk assessment and mitigation

Risk	Impact	Likelihood	Mitigation actually applied
Public dataset has no ready target column	High	Certain	Targets built from published formulas; method documented in Chapter 6
Model performs worse than a plain formula	Medium	Medium	Formula baseline computed and reported next to model results
Training and serving preprocessing differ	High	Medium	One shared Python package used by both; verified by a reload test
Food catalogue too small for the variety rule	Medium	High	USDA data and 52 Sri Lankan items added; coverage measured
Library version mismatch breaks saved models	High	Medium	Occurred in practice; versions now pinned to scikit-learn 1.6.1
Time overrun on the web application	Medium	Medium	Machine learning finished first; web layers built on a fixed API contract

3.0.3 SWOT

Table 3.2 SWOT analysis

Strengths	Weaknesses
Combines prediction, recommendation and long-term planning in one free system. Shared code between training and serving removes a whole class of bug. 263 automated tests. Includes local Sri Lankan food.	Only 973 training records. Fitness goals in the training data are assigned, not self-reported. No allergy or medical condition handling. Not yet deployed to a public server.
Opportunities	Threats
Add Sinhala and Tamil language support. Connect to fitness trackers for automatic activity data. Add photo-based food logging. Expand the local food set with laboratory-measured values.	Established apps have far larger food databases. Users may not log weight often enough for the adjustment loop to help. Nutrition guidance carries responsibility if users treat it as medical advice.

3.0.4 PESTEL Analysis

Table 3.3 PESTEL analysis

Factor	Relevance to this project
Political	No specific regulation on nutrition guidance software in Sri Lanka, but the system avoids making medical claims.
Economic	Dietitian consultations are expensive for many gym users; a free tool lowers that barrier.
Social	Gym membership and interest in fitness are growing, and diet advice is commonly taken from social media.
Technological	Free cloud services and open-source machine learning libraries make this type of system possible at no cost.
Environmental	Minimal. Training used a CPU runtime for about ten minutes rather than sustained GPU compute.
Legal	Health data is personal data, so access control and a clear disclaimer were treated as requirements, not extras.

3.0.5 Life Cycle Model

An iterative and incremental model was used rather than Waterfall. The reason is practical: the machine learning results could not be predicted in advance. The first modelling attempt gave the opposite result to what was expected and had to be investigated and redone. A Waterfall plan with a single fixed design phase would not have allowed that.

Each increment delivered something testable, from data preparation through to the user interface, with tests written alongside.

3.1.1 Time Plan

The work was planned across twelve weeks in five phases. Planning and the literature review ran in weeks 1 to 4. Data preparation, model training and evaluation ran in weeks 4 to 7. The database, the machine learning service, the backend and the front end were built in weeks 8 to 10. Testing was carried out in week 11 and documentation in week 12. The full schedule is shown as a Gantt chart before the references.

Table 3.4 Project phases

Phase	Weeks	Main deliverable
Planning and proposal	1 to 2	Approved proposal and project plan
Literature review	3 to 4	Review of existing systems and techniques
Data preparation	4 to 5	Cleaned dataset and constructed targets
Model training and evaluation	6 to 7	Two trained models with metrics
Database, API and front end	8 to 10	Working three-tier application
Testing	11	263 automated tests
Documentation	12	This dissertation

4.0 Requirement Gathering and Analysis

4.2 Requirement Gathering and Analysis

4.2.1 Requirement Gathering technique used for the project

Three techniques were used to gather requirements for this project.

Document analysis. The approved proposal was the primary source. Its ten functional and ten non-functional requirements were carried into the build, and each is traced to code in Chapter 6.

Existing system analysis. Four products were examined in Chapter 2: MyFitnessPal, Cronometer, MacroFactor and free TDEE calculators. What they lack produced the requirements for meal planning and automatic readjustment.

Dataset analysis. The public datasets were examined before the requirements were fixed, because they limit what can be built. This is how body fat percentage entered the design: the gym dataset carries a measured column, which allows the more accurate Katch-McArdle equation. That field was not in the proposal and was added after seeing the data.

4.2.2 Questionnaire

A questionnaire was designed to test the assumptions taken from the proposal and the existing system review against real gym users. The questions are mostly closed, so the answers convert directly into design decisions such as which profile fields to ask for.

The target population is adults training at a gym at least twice a week in the Colombo area, with a sample of 15 to 20 respondents. The instrument has 10 questions in four sections, reproduced in Appendix 1.

Table 4.1 Structure of the questionnaire

Section	Questions	What it establishes
Training profile	Q1 to Q2	Whether the assumed activity range (2 to 5 sessions a week) is realistic
Current knowledge	Q3 to Q4	Whether users already know their calorie and protein targets, and where from
Existing tools	Q5 to Q6	Which apps they have used, why they stopped, and whether planning ahead is wanted
System preferences	Q7 to Q10	Weighing frequency, importance of familiar food, willingness to enter body fat

Responses will be analysed with counts and percentages for the closed questions and grouped into themes for the free-text ones. Two questions matter most to the design. Q8 asks how important familiar food is, which tests the regional catalogue work in Chapter 6. Q9 asks whether users would enter a measured body fat percentage, which decides whether that field stays optional.

Status. The instrument and participant information sheet are complete and are in Appendix 1, with the results table laid out ready to receive the responses. Distribution is the remaining step, so the requirements in section 4.3 rest on the proposal, the existing system review and the dataset analysis, with the questionnaire serving as confirmation.

4.2.3 Interview

Interviews cover what a questionnaire cannot. A closed question can establish that somebody abandoned an app but not why, and the reason is what changes the design. A semi-structured format keeps the same six topics across every participant while allowing follow-up questions.

Three participants give different viewpoints: a gym instructor who advises members on diet, a member training for muscle gain, and one training for fat loss. Three is small, but the purpose is depth; the questionnaire covers breadth.

Table 4.2 Interview topics and the design question each addresses

Topic	Design question it answers
How members currently decide what to eat	Is the problem statement in section 1.2 accurate?
Experience of existing apps	Which features are missing rather than merely disliked?
Reaction to an eight-week plan	Is a long plan useful, or does it feel rigid?
Attitude to logging weight regularly	Is the FR8 and FR9 adjustment loop realistic in practice?
Importance of familiar local food	Does the regional catalogue work justify its cost?
Trust in an automated recommendation	How prominent must the medical disclaimer be?

Recordings will be analysed thematically: transcribe, code each answer against the six topics, then report agreement and difference across the three participants. Status: the guide and consent form are complete and in Appendix 1 with a recording template; conducting the interviews is the remaining step.

Summary of the Interview and Questionnaire

The two instruments work together: the questionnaire measures how widely a view is held, the interviews explain why. Both are checked against the existing system review in Chapter 2, so no requirement rests on one type of evidence.

The requirements in section 4.3 come from the proposal, the existing system review and the dataset analysis, with Table 4.3 recording any change made after the proposal. Once collected, the responses will either confirm these requirements or show where the assumptions were wrong; that comparison is the first item of future work in section 8.2.

4.3 Functional and Non-Functional Requirements

4.3.1 Functional Requirements

Table 4.3 Functional requirements and where they are implemented

ID	Requirement	Implementation	Status
FR1	User registration and authentication	Supabase Auth with JWT; guard middleware on all /api routes	Done
FR2	User profile management	profiles table; POST and GET /api/profile	Done
FR3	BMI calculation	Computed server-side on every weight entry; WHO categories	Done
FR4	Calorie requirement prediction	Random Forest model served by POST /predict	Done
FR5	Protein requirement prediction	Second Random Forest model, same pipeline	Done
FR6	Personalised meal recommendation	Content-based engine; GET /api/recommendations	Done
FR7	Two-month meal plan generation	Eight-week planner; POST /api/mealplan/generate	Done
FR8	Progress monitoring	weight_logs table; trend charts on dashboard	Done
FR9	Recommendation adjustment	Drift check at 7 percent; prompts the user, never overwrites	Done
FR10	Dashboard and reporting	GET /api/dashboard returns BMI, targets, week plan and trends	Done

Two input fields were added after the proposal, because the dataset supported them and they improve accuracy: body fat percentage (optional, estimated when not supplied) and typical session length in hours, which is needed to work out the activity multiplier.

4.3.2 Non-functional Requirement

Table 4.4 Non-functional requirements

NFR	Requirement	How it was met	Evidence
NFR1	Performance	Async FastAPI endpoints; database indexes on user_id and meal_type; response time header	Prediction returns in well under a second in testing
NFR2	Reliability	Formula fallback if a model file fails to load; consistent error middleware	Fallback path covered by API tests
NFR3	Security	Row Level Security on all six tables; JWT on every route; ML service not public	Verified by an anonymous-client check script
NFR4	Usability	Single onboarding form; clear disclaimer; keyboard accessible interface	WCAG 2.1 AA measures applied
NFR5	Scalability	Stateless ML service; database scales separately	Design property; not load tested
NFR6	Availability	Free-tier hosting planned	Not deployed; runs locally only
NFR7	Maintainability	Shared package; modular route folders; 263 tests	Full suite passes
NFR8	Compatibility	Responsive Tailwind layout; REST/JSON API	Production build succeeds
NFR9	Data privacy	No personal data stored in the ML service	Service is stateless by design
NFR10	Accuracy	MAE, MSE, RMSE, R-squared with 5-fold cross-validation	Reported in Chapter 7

Two of these were not fully verified. Availability was not tested because the system was not deployed to a public server, and scalability was not load tested. Both are listed honestly in the limitations in Chapter 8.

5.0 System Design

5.1 Architecture Diagram

The system uses a layered architecture with a separate machine learning microservice. Figure 5.1 shows the four runtime layers and the offline training environment.

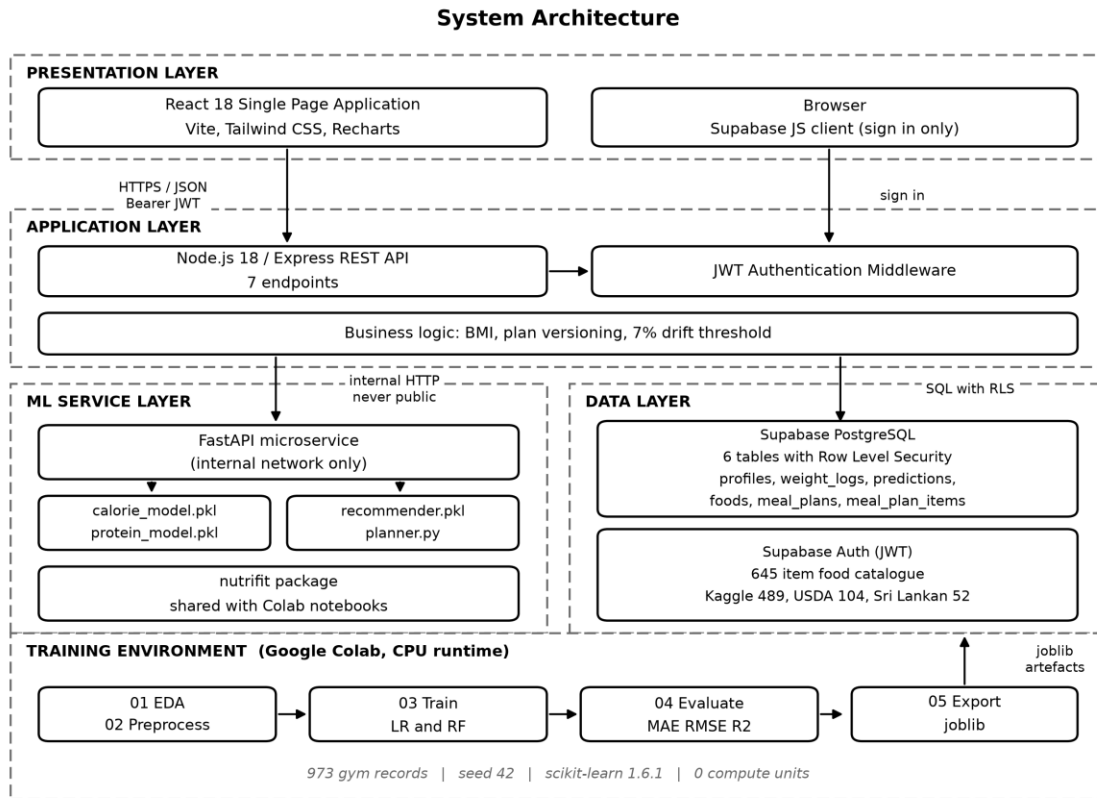


Figure 5.1 System architecture

The important detail is the arrow from the training environment into the ML service layer. The same Python package, called nutrisfit, is used by the Colab notebooks during training and by the FastAPI service at prediction time. The preprocessing steps therefore cannot drift apart between the two.

5.2 ER Diagram

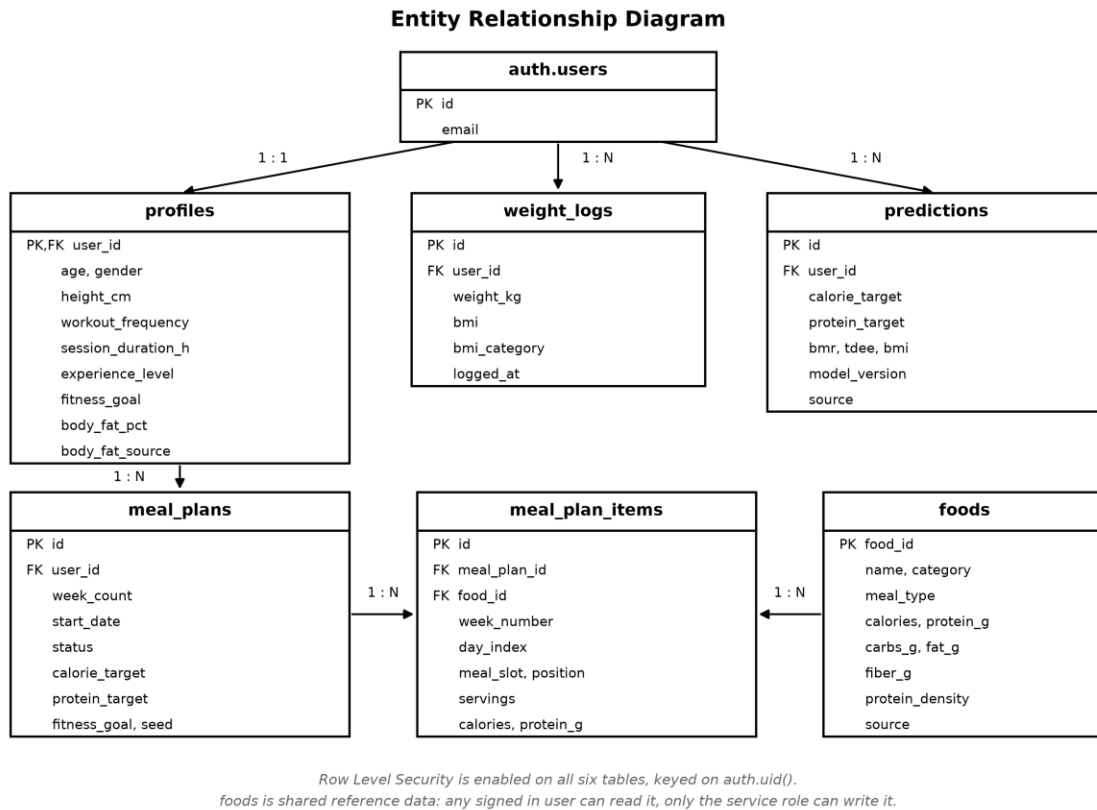


Figure 5.2 Entity relationship diagram

Height is stored on the profile because it does not change, while weight is stored as a series in weight_logs because it does. This separation is what makes the progress chart and the adjustment loop possible. A partial unique index allows only one active meal plan per user; older plans are marked superseded rather than deleted, so history is kept.

5.3 UML Diagrams

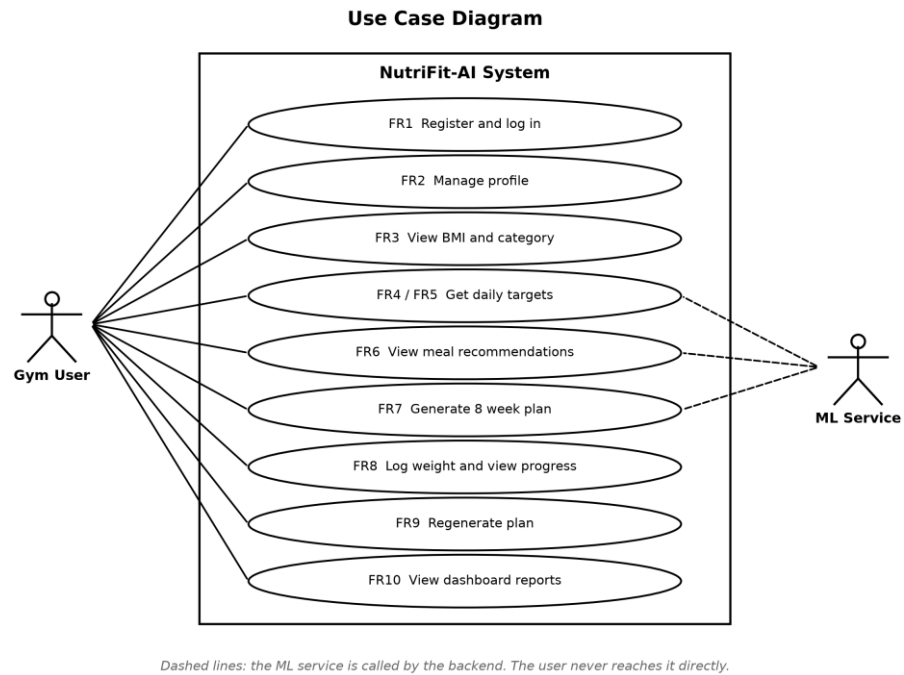


Figure 5.3 Use case diagram

There is one human actor. The machine learning service is shown as a system actor because the backend calls it; the user never reaches it directly.

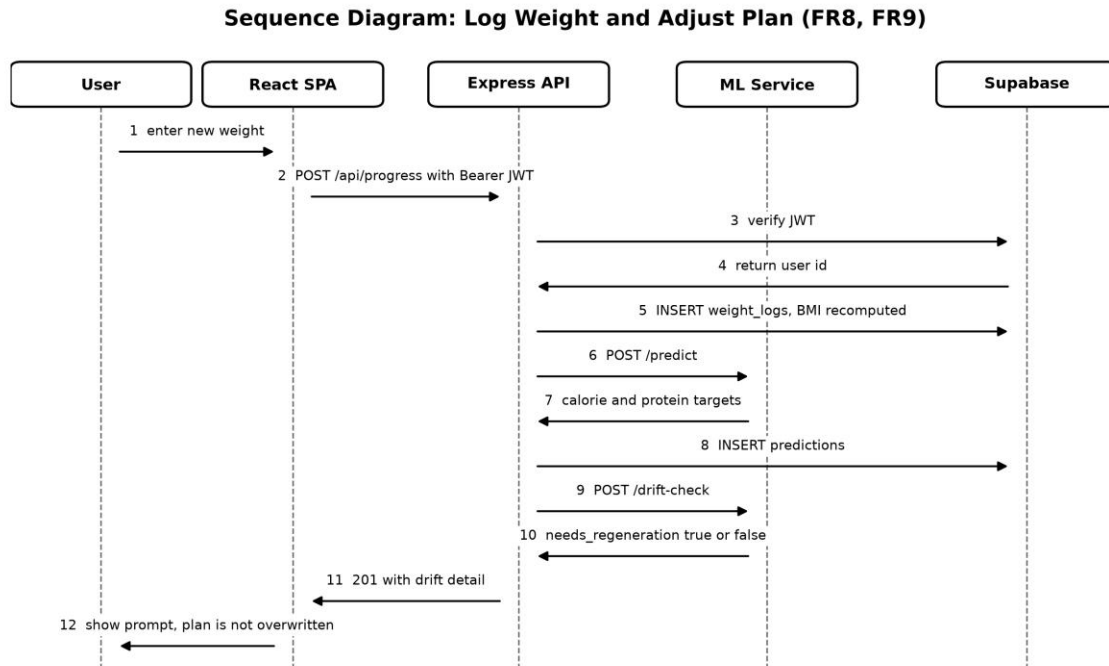


Figure 5.4 Sequence diagram for logging weight and adjusting the plan

This sequence covers FR8 and FR9. Step 12 is the design decision worth noting: the system shows a prompt and lets the user decide. It does not silently replace a plan the user may be halfway through.

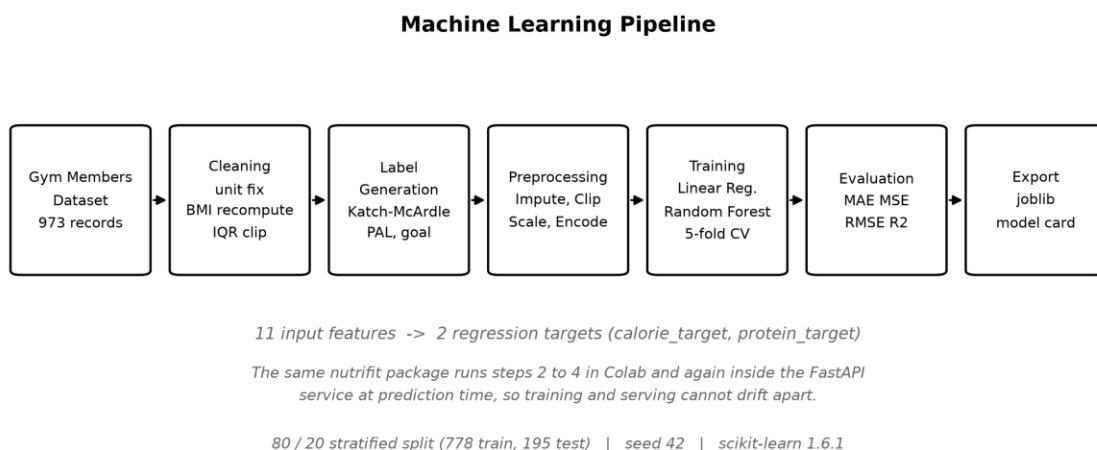


Figure 5.5 Machine learning pipeline data flow

5.4 Wireframe Diagram

The wireframes below match the screens that were actually built.

Wireframes: Authentication and Profile Setup

Login / Register

NutriFit-AI

[Sign in]

[Create account]

Email

Password

At least 8 characters

Sign in

Medical disclaimer:
guidance only, not medical advice

Profile / Onboarding (FR2)

NutriFit-AI

DashboardMeal PlanProgressSign out

Set up your profile

About you

Age

Gender

Height (cm)

Weight (kg)

Body fat % (optional)

Training

Workout days / week

Session length (h)

Experience level

Fitness goal

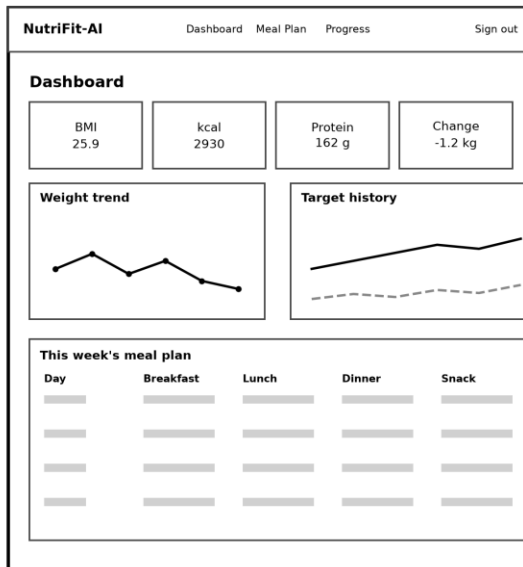
Create profile and calculate targets

Figure 5.6 Wireframes: authentication and profile setup

27

Wireframes: Dashboard and Meal Recommendations

Dashboard (FR10)



Meal Recommendations (FR6)

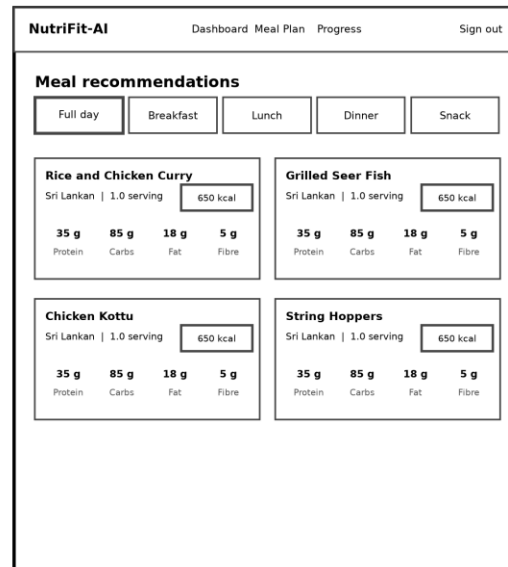
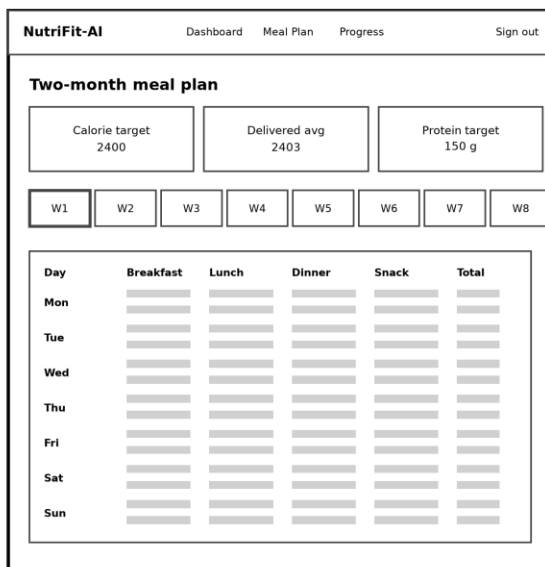


Figure 5.7 Wireframes: dashboard and meal recommendations

Wireframes: Two-Month Meal Plan and Progress

Two-Month Meal Plan (FR7)



Progress (FR8 and FR9)

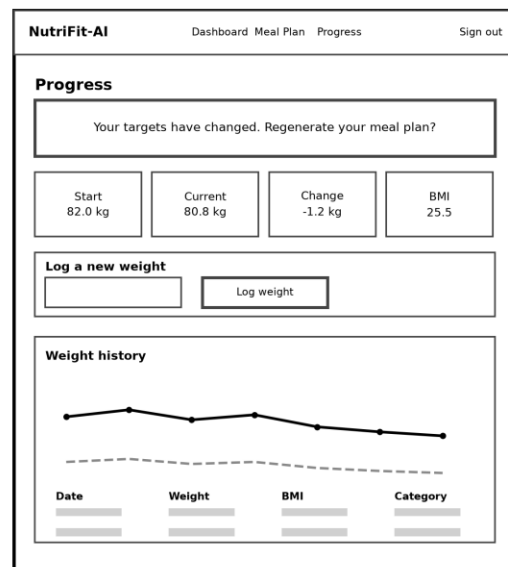


Figure 5.8 Wireframes: two-month meal plan and progress

6.0 Implementation

6.1 Technology Stack

Table 6.1 Technology stack

Layer	Technology	Why it was chosen
Front end	React 18, Vite, Tailwind CSS, Recharts	Component model suits a dashboard; Recharts renders the trend charts
Backend	Node.js 18, Express, Zod, Helmet	Specified in the proposal; Zod validates input, Helmet sets security headers
ML service	Python 3.12, FastAPI, scikit-learn 1.6.1, joblib	scikit-learn only runs in Python; FastAPI validates with Pydantic
Database	Supabase PostgreSQL with Row Level Security	Gives authentication and database together, with access control in the database
Training	Google Colab, CPU runtime	Free; replaces Jupyter from the proposal
Testing	pytest, Jest, Supertest	Standard for each language

FastAPI was added and is not named in the proposal. It is needed because the proposal itself requires the trained model to be integrated with the Node.js backend through APIs, and this service is that integration point.

One practical lesson came from library versions. The models were trained in Colab with scikit-learn 1.6.1 but the local machine had 1.9.0. Loading the saved model failed with a missing internal class error. Saved scikit-learn pipelines depend on private classes that change between versions, so the serving environment must match the training environment. The version is now pinned in both requirements files and recorded in the model card.

6.2 Design patterns

Table 6.2 Design patterns used

Pattern	Where it is used	Benefit
Pipeline	sklearn Pipeline holds preprocessing and the model together	The exported file contains both, so serving cannot skip a step
Repository	Supabase client calls kept inside route modules	Database access is in one place per resource
Middleware	JWT auth, validation, error handling in Express	Cross-cutting concerns are written once
Factory	createApp() builds the Express app	Tests mount the app without opening a network port
Strategy	BMR equation chosen by whether body fat is known	Katch-McArdle when measured, Mifflin-St Jeor otherwise
Facade	The nutrifit package hides the calculation	Notebooks and the API call the same simple

Pattern	Where it is used	Benefit
	details	functions

6.3 Implementation of the program

6.3.1 Datasets used

Three data sources were used. The Gym Members Exercise Dataset from Kaggle provided 973 records with age, gender, weight, height, session duration, workout frequency, experience level and body fat percentage. The Daily Food and Nutrition Dataset from Kaggle provided the meal items with their macronutrients. USDA FoodData Central provided additional whole foods.

Six rows in the food file were malformed, with more fields than the header allowed. Rather than dropping the file, a tolerant reader was written that repairs those rows and records how many were repaired in the data quality report.

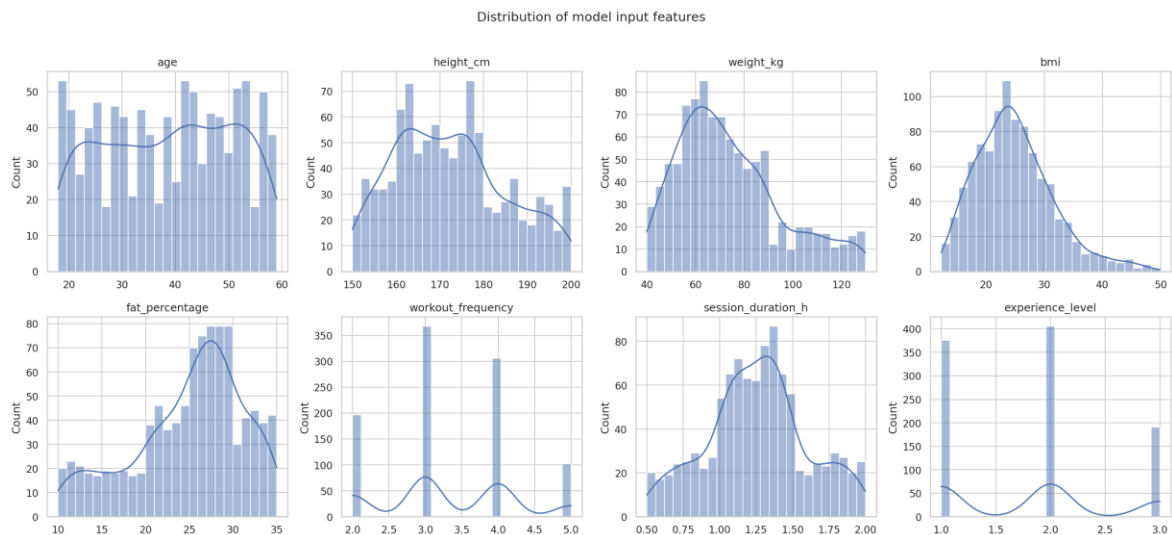


Figure 6.1 Distribution of the model input features

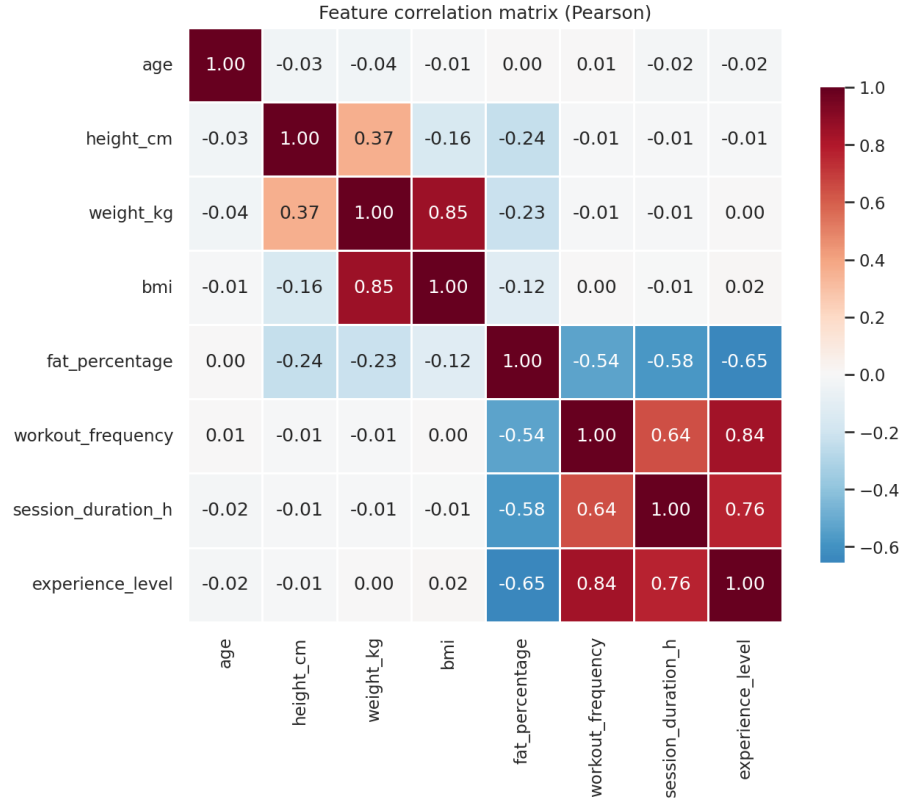


Figure 6.2 Correlation between features

6.3.2 Building the training targets

No public dataset contains a column saying what a person's correct daily calorie target should be. This is the central methodological problem of the project, and the solution needs to be stated clearly.

The targets were constructed from published formulas in four steps. First, BMR is calculated. Where the dataset gives a measured body fat percentage, the Katch-McArdle equation is used, which works from lean body mass. Otherwise Mifflin-St Jeor is used and body fat is estimated with the Deurenberg equation. Second, BMR is multiplied by an activity factor derived from weekly training hours. Third, a goal factor is applied: roughly a 20 percent deficit for fat loss, maintenance, or about a 12 percent surplus for muscle gain. Fourth, protein is set from body weight using ISSN coefficients.

Three sources of variation were added on purpose. Fitness goals are sampled from a probability distribution that depends on BMI and body fat, so a heavier user is more likely to be assigned fat loss. The goal coefficients are sampled within their published ranges rather

than fixed, because the literature gives ranges. Finally Gaussian noise of 3 percent on calories and 2.5 percent on protein is added.

The noise is important and is not there to make the task harder for no reason. Without it, Linear Regression would recover the formula almost exactly and report an R-squared close to 1.0, which would look suspicious and prove nothing. The noise sets a theoretical ceiling on R-squared of 0.9891 for calories and 0.9943 for protein, and these ceilings are reported next to the model results so the remaining error can be identified as irreducible.

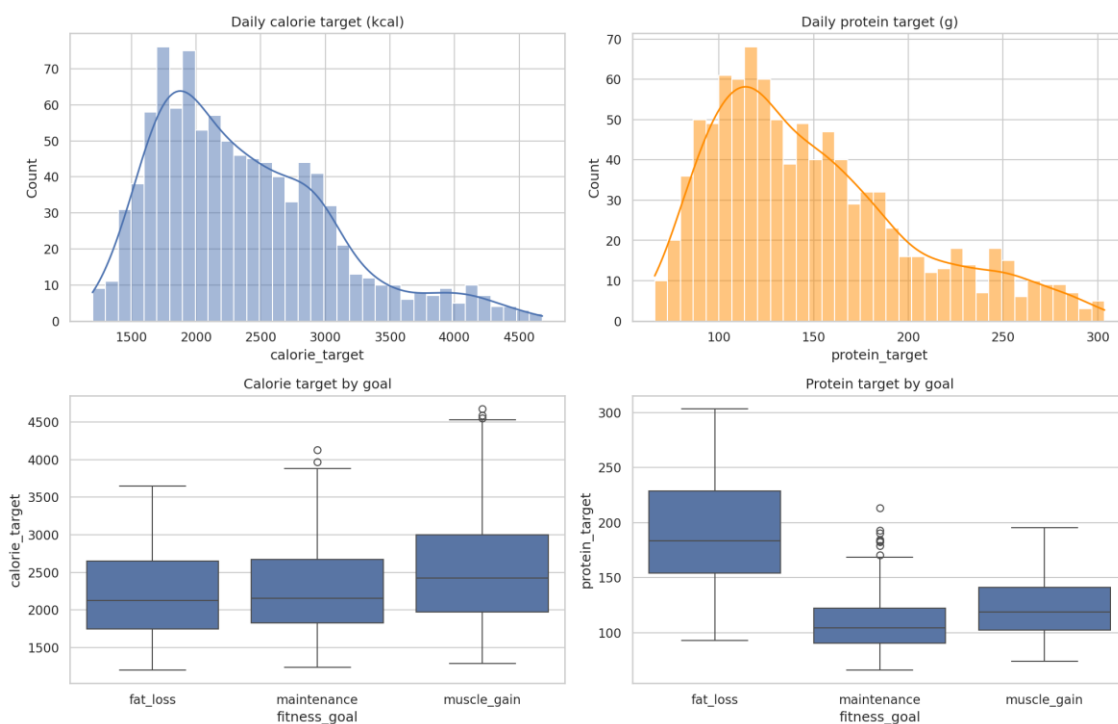


Figure 6.3 Distribution of the constructed calorie and protein targets

6.3.3 Preprocessing pipeline

Preprocessing is a single ColumnTransformer fitted inside the same Pipeline as the model. Numeric features are median-imputed, clipped by interquartile range to catch entry errors such as a 700 kilogram weight, and standardised. Gender and goal are one-hot encoded; activity level is ordinal encoded.

Fitting inside the Pipeline has two effects. The exported file contains preprocessing and the model together, so the API cannot forget a step. And cross-validation refits the scaler on each training fold only, so no information leaks from the test fold.

6.3.4 Recommendation engine

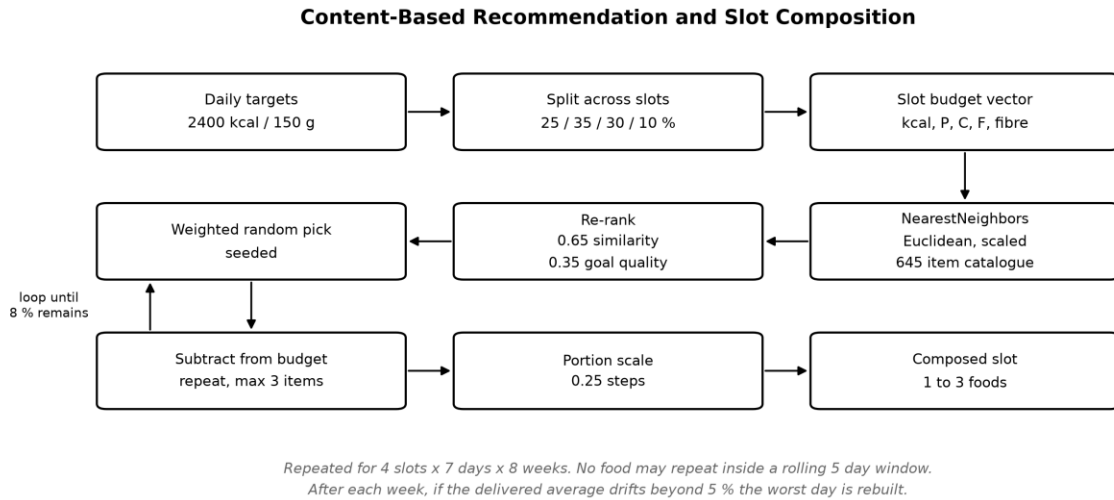


Figure 6.4 Content-based recommendation and slot composition

Euclidean distance is used rather than cosine similarity. Cosine ignores magnitude, so it would treat a 200 kilocalorie meal and a 900 kilocalorie meal with the same macro ratio as identical. For meal planning the absolute amount is the whole point.

The first version of this engine recommended one food per meal slot and failed. The median breakfast item in the data is around 360 kilocalories, while a user needing 2,400 kilocalories a day needs about 600 at breakfast. Measured across a full plan, the system delivered 26 percent fewer calories than the target. The fix was to build each slot from one to three items, then apply a single portion multiplier in quarter-serving steps. After this change the same test produced weekly errors within 0.4 percent.

6.3.5 Meal plan generator

The planner produces 8 weeks by 7 days by 4 slots under two rules. A food may not repeat within a rolling five-day window. And after each week, if the delivered average drifts beyond 5 percent, the worst day is rebuilt with a corrected budget. This is a greedy repair rather than full optimisation, a deliberate trade-off for speed.

6.3.6 Cultural bias in the food catalogue

After the catalogue was built, an audit was run to count how many items matched common Sri Lankan food terms. The result was that hopper, roti, dhal, sambol and kottu each returned zero matches, and curry returned one. Sample recommendations included churros, marshmallow and potato salad.

For the stated target user, a gym-going adult in Sri Lanka, this is a biased system. The macros would be correct but the food would not be eaten, which is an adherence failure. A reference dataset of 52 Sri Lankan items was compiled and added, covering all four meal slots and including vegetarian and high-protein options. After this the same audit returned 27 matches for rice, 14 for curry, 4 for hopper and 3 for dhal, and regional foods make up between 15 and 28 percent of generated plan items.

The macro values for these items are compiled estimates for typical local portions, cross-referenced against USDA values for the ingredients. They are not laboratory measurements, and this is stated in the dataset documentation.

6.3.7 API design

Table 6.3 Backend API endpoints

Method	Endpoint	Purpose
POST	/api/profile	Create or update the profile and log the first weight
POST	/api/predict	Call the ML service and store the prediction
GET	/api/recommendations	Meal suggestions for one slot or a full day
POST	/api/mealplan/generate	Generate and store an eight-week plan
GET	/api/mealplan/active	Fetch the current plan and its items
POST	/api/progress	Log weight, recalculate, and run the drift check
GET	/api/dashboard	One aggregated call for the dashboard

7.0 Testing and Validation

Test Plan

Testing was carried out at four levels: unit tests for the calculations and helpers, integration tests for the API endpoints, model evaluation for the machine learning, and manual system testing of the full user journey. All automated tests are run from the command line and all currently pass.

Table 7.1 Automated test summary

Test suite	Tool	Tests	Result
ML package (nutrition, labels, pipeline, recommender, planner, validation, local foods)	pytest	189	Pass
ML microservice API	pytest and FastAPI TestClient	37	Pass
Node.js backend	Jest and Supertest	37	Pass
Front end production build	Vite	Build	Pass
Total automated tests		263	Pass

7.1 Model Evaluation Results

Both models were trained on 973 records using an 80/20 stratified split, giving 778 training and 195 test records, with 5-fold cross-validation. The random seed was fixed at 42 so the results reproduce exactly.

Table 7.2 Model comparison on the held-out test set and by cross-validation

Target	Model	MAE	MSE	RMSE	R2	CV MAE (mean +/- sd)
Calories	Linear Regression	112.14	23858.30	154.46	0.9564	114.15 +/- 5.17
Calories	Random Forest	107.61	23820.63	154.34	0.9564	96.16 +/- 3.97
Calories	Formula baseline	54.55	-	-	0.9891	-
Protein	Linear Regression	8.71	118.94	10.91	0.9552	8.86 +/- 0.42
Protein	Random Forest	8.37	108.97	10.44	0.9590	8.72 +/- 0.49
Protein	Formula baseline	2.97	-	-	0.9943	-

Random Forest was selected for deployment for both targets, chosen on cross-validated MAE rather than the single test split, because one split is too noisy to decide on.

The formula baseline scores better than both models, but this is expected. The formula is given the activity multiplier directly, whereas the models must infer it from raw inputs. Including the row is more honest than omitting it.

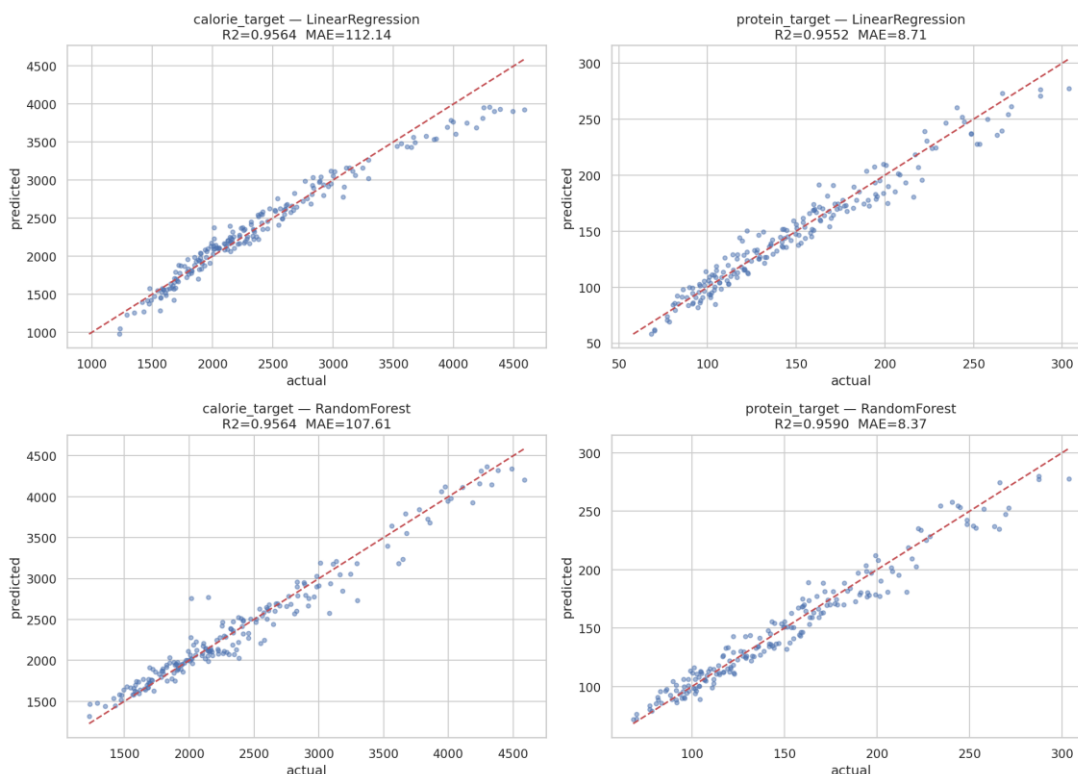


Figure 7.1 Predicted against actual values for both models and targets

7.2 The Learning Curve Experiment

The proposal predicted Random Forest would outperform Linear Regression. On the final dataset it does, but during development an earlier run produced the opposite result and this needed investigation rather than acceptance.

That earlier run used synthetic stand-in data, created so the pipeline could be tested before the real files arrived. Random Forest proved completely insensitive to its hyperparameters there, which pointed at the data rather than the tuning: the synthetic values came from smooth Gaussian distributions with no body-composition signal, so there was no non-linear structure for a tree ensemble to find. Every script now prints a warning when synthetic data is active, and only results from the real files are reported.

To test the explanation properly, a learning curve was computed on the real data by training both models on increasing fractions of it.

Table 7.3 Cross-validated MAE against training set size (calorie target)

Training samples	Linear Regression	Random Forest	Gap (RF - LR)
97	124.58	229.57	+104.99
194	120.38	167.37	+46.98
340	118.94	144.92	+25.98
486	118.19	124.63	+6.44
681	114.28	112.66	-1.62
827	111.98	100.76	-11.21
973	114.15	96.46	-17.69

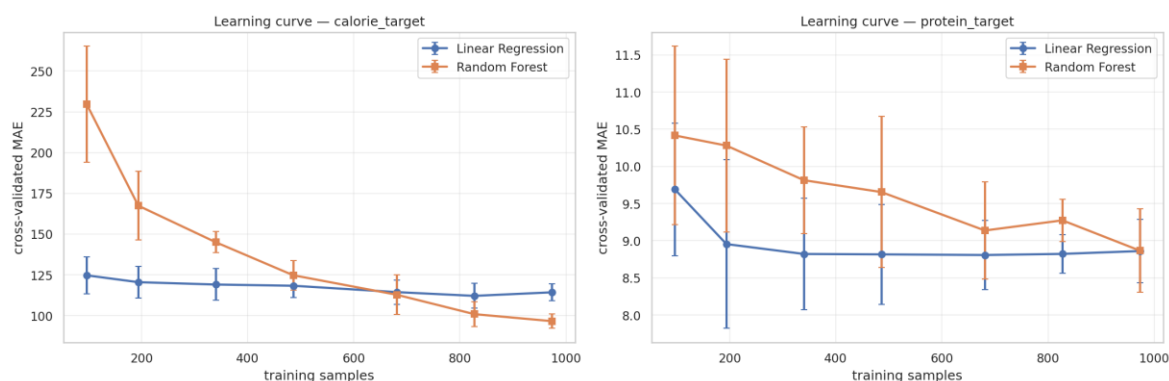


Figure 7.2 Learning curves for both models and both targets

The pattern is clear. Random Forest starts much worse than the linear baseline and improves steadily as data is added, crossing over at around 680 samples. Linear Regression is flat from roughly 200 samples onward, because it has already extracted everything its form can represent. On the protein target the two are still converging at 973 records, with almost no gap left.

This is a textbook bias-variance result. A tree ensemble approximates a smooth relationship with piecewise-constant steps and needs far more data to do it well. The dataset size, not the algorithm choice, is the binding constraint here. That is a more useful conclusion than simply stating that Random Forest won.

7.3 Feature Importance

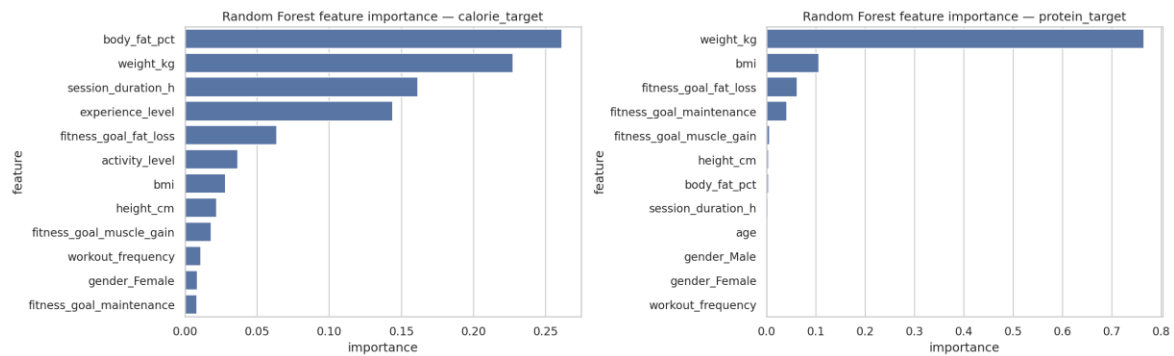


Figure 7.3 Random Forest feature importance

For calories the strongest features are body fat percentage (0.263) and weight (0.226), followed by session duration (0.161) and experience level (0.143). This matches the nutrition science, because body fat and weight determine lean mass, which drives BMR, and training volume drives the activity multiplier.

For protein, weight alone accounts for 0.766 of the importance, with fitness goal next. This is exactly right, since the protein target is body weight multiplied by a goal coefficient. The model recovering this relationship from raw inputs is a good sign that the pipeline works.

7.4 Meal Plan Validation

The eight-week planner was tested against a 2,400 kilocalorie and 150 gram protein target.

Table 7.4 Eight-week meal plan accuracy

Week	Mean daily kcal	Calorie error	Mean daily protein	Protein error	Unique foods
1	2408.9	+0.37 %	149.3 g	-0.50 %	43
2	2399.6	-0.02 %	148.6 g	-0.91 %	40
3	2406.5	+0.27 %	149.6 g	-0.25 %	43
4	2397.1	-0.12 %	146.4 g	-2.38 %	44
5	2402.1	+0.09 %	149.7 g	-0.17 %	41
6	2403.4	+0.14 %	150.2 g	+0.15 %	44
7	2392.3	-0.32 %	151.1 g	+0.71 %	44
8	2396.3	-0.15 %	148.3 g	-1.12 %	45

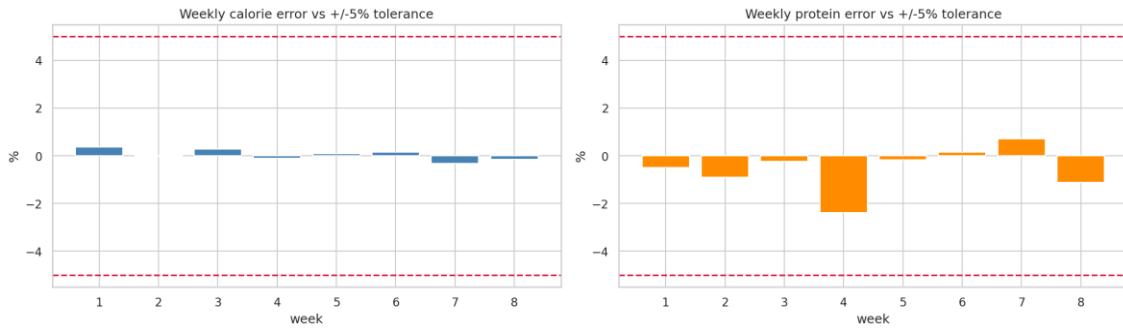


Figure 7.4 Weekly meal plan error against the +/-5 % tolerance

Every week is well inside the 5 percent tolerance, with the largest calorie error at 0.37 percent. The variety rule was satisfied with zero violations.

Test Cases

Table 7.5 Selected test cases

ID	Test	Expected	Actual	Result
TC01	BMI for 80 kg at 180 cm	24.69	24.69	Pass
TC02	Mifflin-St Jeor for 80 kg, 180 cm, 30 y, male	1780 kcal	1780 kcal	Pass
TC03	Katch-McArdle at 15 % body fat, 80 kg	1838.8 kcal	1838.8 kcal	Pass
TC04	Fat loss target below maintenance below muscle gain	Ordered	Ordered	Pass
TC05	Calorie floor never below 1.1 x BMR	Floor applied	Floor applied	Pass
TC06	Prediction with a missing field	422 error	422 error	Pass
TC07	Prediction with body weight of 900 kg	422 error	422 error	Pass
TC08	API access without a token	401 error	401 error	Pass
TC09	Anonymous read of the profiles table	No rows	No rows	Pass
TC10	Same seed produces the same meal plan	Identical	Identical	Pass
TC11	Model reload after export	Prediction in range	2930 kcal	Pass
TC12	Shuffled column order at prediction time	Same result	Same result	Pass

7.5 A Bug Found by Testing

One test was written to check that asking for each meal slot separately would add up to the full daily target. It failed, returning 5,920 kilocalories instead of 2,400. The cause was that the budget function always redistributed the remaining daily allowance across the later slots, so asking for lunch on its own returned 1,120 kilocalories rather than 840. The behaviour was correct when walking through a day in order but wrong for a standalone request. The function

now takes an explicit flag for the two cases. This is a good example of a test catching a defect that manual use would not have revealed.

7.6 What Was Not Tested

Being clear about the limits of the testing matters as much as reporting the results.

- User acceptance testing with real gym users was not carried out.
- Load and performance testing were not carried out, so the performance and scalability requirements are supported by design rather than measurement.
- The interface was not tested with a real screen reader, although the accessibility markup follows WCAG 2.1 AA.
- The system was not deployed to a public server.
- The optional BMI cross-check against the UCI obesity dataset is implemented and unit tested, but was not run because the dataset was not downloaded.

8.0 Conclusion

8.1 Conclusion

This project set out to build a working web application that gives gym users personalised nutrition guidance, and that objective was met. All ten functional requirements from the proposal are implemented and traced to code in Chapter 6. The system predicts calorie and protein needs with a cross-validated error of 96.16 kilocalories and 8.72 grams, recommends real meals, produces an eight-week plan whose weekly calorie delivery stays within 0.4 percent of target and whose protein stays within 2.4 percent, and prompts the user to update when their weight changes.

Five things in this project go beyond standard student work, and these are what I would put forward as its contribution.

- Shared code package. The same Python module preprocesses data during training in Colab and during prediction in the API, so train-serve skew is prevented by structure rather than by discipline.
- Honest treatment of constructed labels. The injected noise is documented and the resulting theoretical R-squared ceiling is reported beside the model scores, so a reader can see how much error is irreducible.
- Learning curve experiment. Rather than asserting Random Forest is better, the project measured how the gap changes with dataset size and located the crossover at about 680 records.
- Meal slot composition. One food per slot under-delivered by 26 percent; building each slot from one to three items with a quarter-serving multiplier brought weekly error inside 0.4 percent.
- Cultural bias audit. Measuring the bias, correcting it with a 52-item local dataset, then measuring again turns a limitation into evidence.

8.2 Future Recommendations

- Run a questionnaire and interviews with Sri Lankan gym users, and carry out user acceptance testing. This is the largest gap in the current work.
- Deploy the system to free hosting so availability can be measured, and add load testing.
- Add allergy and dietary restriction fields, which are currently out of scope.

- Collect real self-reported fitness goals instead of assigned ones, and gather more records, since the learning curve shows the models are still improving at 973.
- Add Sinhala and Tamil language support and expand the local food set with laboratory-measured values.
- Test with a real screen reader and add an automated accessibility check to the build.

8.3 Lessons Learned

The most useful lesson was that an unexpected result is worth investigating rather than working around. When Random Forest lost, the temptation was to tune it until it won. Checking whether the hyperparameters mattered at all showed they did not, which pointed at the data and produced a better chapter than tuning would have.

The second lesson was about environments. The scikit-learn version mismatch broke model loading completely and cost time. Pinning versions and recording them in a model card is not bureaucracy.

The third lesson was that automated tests find things manual use does not. The meal slot budget bug returned a plausible-looking number and would probably never have been noticed by clicking around the interface.

The fourth lesson was about assumptions. I expected that mixing USDA per-100-gram data with per-serving data would harm accuracy. Measured, it improved the worst weekly error from 3.80 to 2.88 percent. Similarly, my first attempt at the Sri Lankan food set made fat-loss plans worse, from 1.42 to 4.93 percent error, because every item I added was a rice-based plate with low protein density. Adding ten protein-forward items brought it back to 3.21 percent. In both cases measuring beat guessing.

Gantt Chart

The chart below shows the twelve-week schedule that was followed. The phases are colour grouped: planning and literature review in blue, machine learning work in green, application development in orange, the accessibility and regional food work in purple, testing in red and documentation in grey.

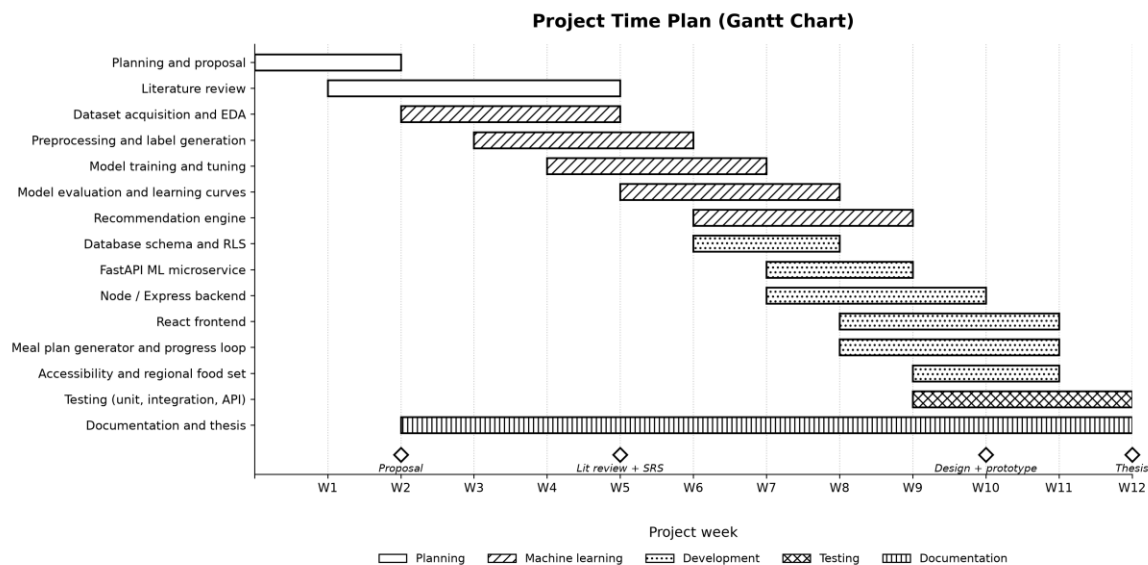


Figure 8.1 Project time plan (Gantt chart)

References

- Deurenberg, P., Weststrate, J.A. and Seidell, J.C. (1991) 'Body mass index as a measure of body fatness: age- and sex-specific prediction formulas', *British Journal of Nutrition*, 65(2), pp. 105-114.
- FAO/WHO/UNU (2004) Human energy requirements. FAO Food and Nutrition Technical Report Series 1. Rome: Food and Agriculture Organization.
- Géron, A. (2022) *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. 3rd edn. Sebastopol, CA: O'Reilly Media.
- Helms, E.R., Zinn, C., Rowlands, D.S. and Brown, S.R. (2014) 'A systematic review of dietary protein during caloric restriction in resistance-trained lean athletes', *International Journal of Sport Nutrition and Exercise Metabolism*, 24(2), pp. 127-138.
- Jäger, R., Kerksick, C.M., Campbell, B.I. et al. (2017) 'International Society of Sports Nutrition position stand: protein and exercise', *Journal of the International Society of Sports Nutrition*, 14(20), pp. 1-25.
- Katch, F.I. and McArdle, W.D. (1996) *Introduction to Nutrition, Exercise and Health*. 4th edn. Baltimore: Williams and Wilkins.
- Mifflin, M.D., St Jeor, S.T., Hill, L.A., Scott, B.J., Daugherty, S.A. and Koh, Y.O. (1990) 'A new predictive equation for resting energy expenditure in healthy individuals', *American Journal of Clinical Nutrition*, 51(2), pp. 241-247.
- Nilashi, M., Ibrahim, O. and Ahmadi, H. (2020) 'Recommendation systems for health and nutrition using machine learning techniques', *Expert Systems with Applications*, 152, pp. 113-128.
- Palechor, F.M. and de la Hoz Manotas, A. (2019) 'Dataset for estimation of obesity levels based on eating habits and physical condition', *Data in Brief*, 25, 104344.
- Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825-2830.
- Raza, S., Ahmed, M. and Khan, A. (2023) 'Artificial intelligence in nutrition recommendation systems: a survey', *IEEE Access*, 11, pp. 65421-65440.
- Ricci, F., Rokach, L. and Shapira, B. (2015) *Recommender Systems Handbook*. 2nd edn. New York: Springer.
- United States Department of Agriculture (2024) FoodData Central. Available at: <https://fdc.nal.usda.gov> (Accessed: 7 August 2026).
- W3C (2018) Web Content Accessibility Guidelines (WCAG) 2.1. Available at: <https://www.w3.org/TR/WCAG21> (Accessed: 7 August 2026).
- World Health Organization (2024) Obesity and overweight. Geneva: WHO. Available at: <https://www.who.int> (Accessed: 7 August 2026).

Appendix 1

Questionnaire

The full instrument is reproduced below. Response counts are recorded in the table that follows once collection is complete (see section 4.2.2 for the current status).

- How often do you train each week, and for how long per session?
- What is your current main fitness goal: fat loss, muscle gain or maintenance?
- Do you currently know your daily calorie target? If yes, where did that number come from?
- Do you know your daily protein target?
- Have you used a nutrition or calorie app before? Which one, and why did you stop?
- How useful would an eight-week meal plan be compared with daily suggestions?
- How often would you be willing to record your body weight?
- How important is it that suggested meals are foods you normally eat?
- Would you enter a body fat percentage if you had one measured?
- What would stop you from following a meal plan given by an app?

Participant information and consent. Respondents are told the purpose of the study, that participation is voluntary, that no identifying information is recorded, and that they may withdraw at any time. Consent is taken on the first page of the form.

Table A1 Questionnaire response summary sheet

Q	Question	Responses
Q1	Sessions per week	
Q2	Typical session length	
Q3	Knows daily calorie target	
Q4	Knows daily protein target	
Q5	Apps used previously and reason for stopping	
Q6	Usefulness of an eight-week plan	
Q7	Willing weighing frequency	
Q8	Importance of familiar food	
Q9	Would enter measured body fat	
Q10	Barriers to following a plan	

Interview Guide

Semi-structured guide used with all three participants. Prompts are used only if the topic is not covered in the opening answer.

- Opening: tell me how you currently decide what to eat on a training day.
- Existing tools: which nutrition apps have you tried? What made you stop?
- Planning: if the system gave you a full eight-week plan, would you follow it, or would that feel too rigid?
- Adjustment: how often would you realistically weigh yourself and record it?
- Food familiarity: how would you feel about a plan built around foods you do not normally eat?
- Trust: how much would you trust a calorie target produced automatically, and what would make you trust it more?

Table A2 Interview response recording sheet

Topic	P1 instructor	P2 muscle gain	P3 fat loss
Deciding what to eat			
Existing apps			
Eight-week plan			
Weighing frequency			
Familiar food			
Trust in automation			

Test Case

The selected test cases are given in Table 7.5. The complete automated suite of 263 tests is in the source code under `ml/tests`, `services/ml-service/tests` and `backend/tests`, and is run with `pytest` and `npm test` as described in the project README.

Supervision meeting log sheets

The signed supervision meeting log sheets are attached separately as required by the assessment brief.